

Robotics Security Platform

Interview Preparation — Part 3 of 4: The Brain (Detection, AI, Response & DevSecOps)

Part 2 was the plant floor. This part is everything that watches it and reacts: how raw network traffic becomes numerical features, how four machine-learning models across two detection planes (network traffic and robot motion) decide what is an attack, how the incident-response engine contains a threat without endangering anyone, how vulnerabilities are tracked offline, and how the robot/PLC code itself is gate-checked by a CI/CD pipeline. This is the longest part — it is the "intelligent" half of "intelligent security platform".

WHY IT MATTERS

The spine of this whole part is one pipeline: **traffic** → **features** → **score** → **alert** → **incident** → **response**. Keep that chain in your head and every component below slots into it.

1. The Security zone: passively watching the wire

The SEC zone's job is to **observe** OT traffic and turn it into something the AI can use — without ever touching the plant. It only sees a **mirror** (a copy) of the traffic, so it physically cannot interfere.

1.1 Zeek, Suricata, ntopng — three lenses on the same traffic

Tool	What it is	Its role here
Zeek	A network analyser that understands protocols deeply	Parses Modbus (and DNP3/OPC-UA), and emits per-message feature rows for the ML pipeline.
Suricata	A signature-based IDS	Catches known-bad patterns with rules (e.g. a write to the motor-enable coil from a non-HMI source).
ntopng	A network-flow visualiser	Operator-facing view of who is talking to whom on the network.

WHY IT MATTERS

This is "defence in depth" inside detection itself: Suricata catches the *known*, the ML models catch the *unknown/novel*, and ntopng gives a human the big picture. No single blind spot.

1.2 The clever Zeek rule worth quoting

Zeek's local policy declares a custom alert called **Modbus_Write_From_Outside_OT**. In plain words: **"If you see a Modbus *write* command (function code 5, 6, 15, 16, 22, 23) and the sender is not inside the OT subnet, raise a notice."** That is the single most suspicious thing in this environment — a write to the controller from somewhere it should never come from.

1.3 Shipping the features to the brain

A small program, **feature_pusher.py**, tails Zeek's feature log and pushes each row as JSON onto a Redis list named **lab.modbus.features.raw**. It is written to survive the messy realities: Zeek rotates its log every hour (it re-opens on inode change) and Redis might blip (it backs off and re-seeks so no row is lost).

2. From traffic to features (the 20-number fingerprint)

Machine-learning models do not read network packets; they read numbers. So the pipeline summarises traffic into a **20-number vector** describing one source IP's behaviour over a **5-second window**. The exact same code computes these for training and for live scoring, so the model and the live scorer always agree.

2.1 The 20 features (grouped so they are memorable)

Group	Features	Intuition
Volume	n_msgs, msg_rate, n_unique_funccodes, n_unique_addresses	How much and how varied is the traffic?
Read vs write	n_reads, n_writes, write_ratio, write_read_ratio, bulk_write_ratio	Normal HMLs mostly read; attacks write — especially bulk writes.
Where from	ot_origin, n_external_writes	The key signal: writes coming from outside the OT zone.
Errors	n_exceptions, exception_rate	Attackers poke at things and cause error storms.
Targets	mean_address, std_address, mean_quantity, max_quantity	Which registers, and how big the operations are.
Timing	mean_iat_ms, std_iat_ms	Bursts have low, uniform inter-arrival times.
Shape	func_entropy	Attack traffic mixes function codes unusually (high entropy).

SAY THIS IN THE INTERVIEW

If they ask "what is your single best feature?": **n_external_writes** — the count of write commands from a non-OT IP. That is the textbook command-injection / exfiltration signal, and it directly mirrors the Zeek rule from section 1.

2.2 The Redis bus (why a queue sits in the middle)

```
vm-sec                                vm-ai
feature_pusher --RPUSH--> [ lab.modbus.features.raw ] --BLPOP--> feature_consumer
                                                                | scores window
                                                                [ lab.anomaly.events ] <--RPUSH-----+
                                                                |
                                                                +--BLPOP--> alert_bridge --> ai-alerts.json
```

Redis decouples the fast producer (network sensors) from the slower consumer (ML scoring). A sudden traffic burst piles up harmlessly in the queue instead of stalling or crashing the scorer.

3. The machine-learning models (two detection planes)

Detection runs on **two independent planes**. The **network plane** scores each 5-second Modbus window with **three** complementary anomaly detectors (below). The **robot-behavior plane** (section 3.3) watches the robot's own motion with a **fourth** model. Four models, two planes, no single blind spot — if *any* detector flags its input above its calibrated threshold, it counts as anomalous.

3.0 Network plane — three detectors on the 20-feature Modbus vector

Model	How it thinks	What it is best at
Isolation Forest	Randomly "cuts" the data; anomalies get isolated in very few cuts.	Cheaply catching rare <i>combinations</i> of features.
PCA autoencoder	Compresses then reconstructs the vector; big reconstruction error = odd.	Catching windows that do not fit the normal "shape" of data.
TensorFlow autoencoder	A neural network that learns to reconstruct normal traffic.	Catching subtler, non-linear patterns the others miss.

3.1 How they are trained (this is where the rigour shows)

- **Trained on pure normal traffic only** — zero attack data in training. The model learns "normal" so cleanly that anything unusual stands out. (This is what makes it an *anomaly* detector, not a classifier.)
- **Threshold calibrated to the 99th percentile** of normal scores — aiming for at most ~1% false alarms. Crucially the autoencoder thresholds are calibrated against the **live** Zeek pipeline, not just synthetic data, so the model's notion of "normal" matches what it sees in production (no train/serve drift).
- **Quality measured with AUC** on held-out attack data, so the detection performance is actually quantified, not assumed.
- For Isolation Forest specifically: 300 trees for stable scoring, and scores are made always-non-negative so "bigger = more anomalous" is consistent.

SAY THIS IN THE INTERVIEW

"Why anomaly detection instead of signatures?" → "OT attacks often *live off the land* using legitimate protocol commands, so there is no signature to match. A model of *normal* flags malicious-but-novel behaviour — like a register that is normally only read suddenly being written from outside the OT zone."

3.2 Keeping false positives down (4 controls)

A security tool that cries wolf gets switched off. So beyond the calibrated threshold, the consumer adds three more brakes:

- **Two-window debounce** — an alert needs *two consecutive* anomalous 5-second windows. A real attack persists; a one-off blip is ignored.
- **Per-source cooldown** — after alerting on a source IP, it waits ~45 seconds before alerting on it again, so one attack is not 50 alerts.
- **Calibrated (not hard-coded) thresholds** — each model uses its own learned 99th-percentile threshold, with an env override available for noisy links.
- A single attack that trips several detectors is folded into **one incident** (campaign de-duplication), not many.

3.3 Robot-behavior plane — the LSTM autoencoder + physical envelope

The three models above watch the *network*. But an attacker who already controls the PLC can move the robot through *legitimate-looking* commands — so a fourth model watches the **robot's motion itself**. A

passive tap (`joint_telemetry_bridge.py`) mirrors the live `/lab_arm/joint_states` stream to the AI — exactly as Zeek mirrors the network — and `robot_consumer.py` scores it with two layers:

Layer	How it thinks	What it catches
LSTM autoencoder	A recurrent neural net learns the normal pick-and-place motion over a 5 s window of joint angles and velocities, and flags windows it reconstructs badly.	Subtle trajectory tampering — the arm reaching somewhere it never normally goes.
Physical envelope	Deterministic limits calibrated from normal motion: joint speed, position range, jerk, and a frozen-joint check.	Hard faults — over-speed, out-of-range, jitter, or a sensor-frozen joint.

WHY IT MATTERS

Why two layers? The learned model catches the *subtle and novel*; the envelope catches the *physically gross* and is fully explainable ("j1 exceeded its speed limit"). A frozen joint is the neat case: it is *too simple* for the autoencoder to flag, so the deterministic frozen-joint rule catches it instead.

- **Anti-drift by design** — one shared, versioned module (`robot_features.py`, `ROBOT_FEATURE_VERSION`) computes the features for *both* training and live scoring, and the tap emits only raw joint angles (velocities are derived in that one place). A contract test asserts the live windows are byte-for-byte what training saw.
- **Trained on pure-normal motion**, 99th-percentile threshold, **ROC-AUC ~0.99** on held-out behavioral attacks — the same rigour as the network models.
- **Closes the loop** — a robot anomaly raises a robot-behavior-anomaly alert that drives the same incident-response engine, whose containment ladder ends in a latched **E-stop**.

DETECTION CHEAT-SHEET

Key numbers: **network plane** = 5-second window, 20 features, 3 models; **robot plane** = 5-second window of 12 joint channels, LSTM autoencoder + physical envelope. All use **99th-percentile** thresholds, a **2-window** debounce and a **~45 s** cooldown. Sub-5-second detection in the demo.

4. The scoring service (the API in the middle)

`score_service.py` is a FastAPI web service on port 8000. It is the hub the dashboard talks to. Its endpoints group into a few jobs:

Job	Endpoints	What it does
ML scoring	<code>/score</code> , <code>/score/window</code> , <code>/metadata</code> , <code>/health</code>	Score feature vectors on demand; report model info and liveness.
Trend analytics	<code>/api/trend</code> , <code>/api/trend/history</code>	The threat sparkline the dashboard draws.
Incident response	<code>/api/ir/incidents</code> , <code>/pending</code> , <code>/approve</code>	List incidents, list steps awaiting approval, and approve/reject them.
HMI / control	<code>/api/hmi/state</code> , <code>/control</code> , <code>/simulate-button</code> , <code>/logs</code>	Read live PLC state; send start/stop/E-stop/reset; tail service logs.
Security posture	<code>/api/stages/reports</code>	Aggregate the vuln inventory, integrity baseline, and pipeline verdicts.
Demo	<code>/api/demo/inject-attack</code> , <code>/injection-state</code>	Trigger the simulated attack used in the live demo.
Vendor access	<code>/api/vendor/*</code>	Provision time-boxed vendor remote-access sessions (section 7).

4.1 Two security facts to state precisely

- **The API key is fail-closed.** If no key is configured, the server refuses (503) rather than running open; a wrong key gets 401. (It used to "fail open" — run unprotected if the key was empty — and that was fixed.)
- **The browser never holds the key.** The dashboard's nginx proxy injects the secret **X-API-Key** header server-side for every `/api/` call, so the secret is never shipped to the browser.

TRADE-OFF / BE HONEST

Be honest about the architecture smell: `score_service` is a **"god object"** — one ~1,100-line service doing eight jobs (scoring, trends, IR, HMI control, logs, reports, demo, vendor). The code even documents the intended split into `mlapi/irapi/hmiapi/demoapi`. **Most importantly, a pure "monitoring" service should not hold PLC write authority** — in production that control path belongs in an authenticated OT-zone gateway, keeping analytics read-only. Raising this yourself shows you understand IEC 62443 zoning.

5. Incident response (the playbook engine)

When an alert is raised, the **playbook_engine.py** takes over. It matches the alert to a **playbook** (a YAML-like recipe) by category, then runs that playbook's steps in order. The flagship playbook is **pb_modbus_replay** for a Modbus attack.

5.1 Graded containment — the "do no harm" ladder

This is the heart of OT incident response, and the project gets it right. The response is **graded**: gentle automatic steps first, and anything that touches the physical process **waits for a human**.

Grade	Action	Approval	Why
Watch	Increase logging detail; capture forensic evidence.	Automatic	Harmless; you always want the evidence.
Isolate	Firewall-drop the attacker's IP (iptables DROP).	Automatic	Stops the attack at the network without touching the robot.
Slow	Drop the robot to reduced safe speed (~25%).	Human	Changes physical motion — needs a person.
Stop	Assert the safe state / E-stop.	Human	Halts production — only a human should decide that.

WHY IT MATTERS

This is the #1 OT-security principle: **availability is king**. You do NOT let an AI halt a live factory on a hunch — a false alarm that stops a line costs a fortune. So the automatic action ceiling is "isolate the attacker's IP"; the robot keeps running. Slowing or stopping the robot is queued and waits for an operator to click Approve. The project's playbook literally encodes this with `requires_human_approval: true` on the stop step.

5.2 How a human approval works

When the engine hits a step needing approval, it does not run it — it writes the step into a **pending_approvals** queue and pauses that branch. The dashboard shows the pending action. An operator approves via the API (or the **ir-approve** tool), which then runs the queued command, records the result in **incidents.jsonl**, and auto-closes the incident when nothing is left pending.

5.3 Built-in safety against being abused

- Values substituted into commands (like the attacker IP) are **shell-quoted**, and the source IP is checked against a strict numeric pattern — so a forged alert cannot inject shell commands.
- The lifecycle follows **NIST SP 800-61**: detect → analyse → contain → eradicate → recover → lessons-learned. Recovery is a deliberate operator reset, never automatic.
- Other playbooks exist too: one for an unauthorized SROS2 participant, one for an integrity mismatch.

SAY THIS IN THE INTERVIEW

"Containment that does not touch the physical process runs automatically; anything physical needs operator approval. Inputs used in commands are sanitised, and one attack that trips several detectors is de-duplicated into a single incident." That one sentence answers "how do you stop the IR engine being abused?"

6. Vulnerability management (knowing what you have, and what is wrong with it)

You cannot protect what you do not know you have. The SEC zone runs five tools that together answer: what devices exist, what known vulnerabilities they have, has anything changed, and how do we safely push a fix.

Tool	In plain words	Output
inventory.py	Finds every device: passively from Zeek logs, then a gentle rate-limited Nmap, then a Modbus "who are you?" query (function code 43) to fingerprint the PLC vendor/firmware.	inventory.json
cve_correlate.py	Matches each device against an offline hand-curated CVE database (no internet from OT) to list known vulnerabilities.	vulnerabilities.json
integrity_baseline.py	Takes a fingerprint (SHA-256 hashes) of the PLC programs, the SROS2 files, a Modbus snapshot, and which safety services are running.	integrity_baseline.json
baseline_check.py	Compares the live system to the approved baseline and reports drift.	baseline_drift.json
firmware_workflow.py	Drives a safe PLC-program update: stage → validate → schedule → backup → apply → verify → rollback, only inside a maintenance window.	a per-run audit trail

PLAIN-ENGLISH EXAMPLE

Two details interviewers love: (1) the CVE feed is **offline** — a regulated OT site cannot make outbound internet calls, so it consumes a signed advisory snapshot, exactly as this does. (2) The Nmap scan is deliberately **rate-limited and port-restricted**, because a normal aggressive scan can crash fragile old PLC network stacks. "OT-safe scanning" is a real skill.

NOTE

Why fingerprint with Modbus function code 43 (Read Device Identification)? Because it is the *same* query a legitimate engineering tool uses — so you get a trustworthy vendor/product/firmware string with zero risk, rather than guessing from open ports.

7. DevSecOps: gate-checking the robot's own code

Robot and PLC code *is* code, and unsafe code on a robot is a physical hazard. So every change runs through a **six-gate pipeline** before it is trusted. Treat dangerous code like luggage at an airport: it goes through scanners before it is allowed in.

7.1 The six gates

#	Gate	What it blocks
1	PLC lint (plc_lint.py)	Unsigned programs, motion with no E-stop guard, hard-coded passwords, writes to safety outputs from normal code, unbounded loops.
2	HMI lint (hmi_lint.py)	Insecure settings in the operator-screen configuration.
3	SROS2 lint (sros2_lint.py)	Robot-permission policy mistakes.
4	Vulnerability gate (vuln_gate.py)	Any known CVE with score ≥ 7.0 that has no approved, time-boxed exception.
5	Baseline gate (baseline_gate.py)	Any critical configuration drift — or a scan that is stale (> 24 h old).
6	Acceptance gate (acceptance_gate.py)	A change that breaks detection or the safety loop — tested by replaying an attack against the digital twin .

7.2 One engine, three triggers (the clean design)

All gate logic lives in **one script, run_pipeline.sh**. Three different things can trigger it, so a check can never pass in one place and fail in another:

(1) LOCAL	developer runs it before pushing	LAB_GATES=plc,hmi,sros2
(2) GITEA CI	every push/PR -> Actions runs the static gates (1-3) via the same engine	red/green per gate in the Gitea Actions tab
(3) WEBHOOK	a push fires an HMAC-signed webhook (fail-closed) into the lab	-> full in-lab pipeline, signed artifact + verdict

WHY IT MATTERS

"One engine, two triggers" is the industry-standard **"thin CI, fat scripts"** pattern. Gates 4–6 need live lab data (fresh scans, a running digital twin), so they run in-lab, not on a bare CI runner. And you never replay attacks against a producing line — only the Gazebo twin. That is the availability-first mindset again.

7.3 What "signed artifacts" means

When a build passes, the pipeline packs the approved code into a tarball and seals it: a **SHA-256 hash** (a fingerprint — change one byte and it changes) and a **GPG signature** (a cryptographic stamp only the build system can make). Before code is flashed to a robot, you re-check the seal — proving it is the *exact* code that passed every gate and nobody swapped it afterwards.

PLAIN-ENGLISH EXAMPLE

Analogy: the artifact is a sealed bottle of medicine. The hash is the tamper-evident seal (broken = someone opened it). The signature is the pharmacy's official stamp (proves it is genuine, not counterfeit). Together they defeat supply-chain tampering.

TRADE-OFF / BE HONEST

Honest caveat to volunteer: in the shipped config the heavy acceptance gate (6) is skipped for speed (LAB_SKIP_ACCEPTANCE) and the signing key is generated at runtime with no external trust anchor. For production I would enforce every gate on deploy and sign with an HSM/KMS-backed key with verified provenance.

8. Monitoring, vendor access, and the dashboard

8.1 Prometheus + Grafana + a custom exporter

Prometheus scrapes numerical metrics on a schedule; **Grafana** draws them as dashboards. A small zero-dependency **lab_exporter.py** publishes the lab-specific numbers that off-the-shelf exporters do not have:

- anomaly alert counts, the live **safety state** (0 NORMAL / 1 DEGRADED / 2 EMERGENCY), vulnerability counts by severity,
- the last pipeline verdict (pass/fail) and its age, plus open incidents and pending approvals.

8.2 Vendor / remote access (the DMZ jump host)

Outside vendors sometimes need to service the robot, but must never get a direct line into OT.

vendor_access.py provisions a **time-boxed, audited** session (1–8 hours, "read-only" or "maintenance") that lands in **Apache Guacamole** — a browser-based jump host in the DMZ. Every session needs a name, email, and justification, is logged to an audit trail, and the provisioning endpoint now requires the API key.

SAY THIS IN THE INTERVIEW

"Vendors never touch OT directly. They get a brokered, time-limited, audited session through a DMZ jump host — read-only by default. It is the monitored visitor meeting room, not a key to the building."

8.3 The operator dashboard (7 pages)

The React dashboard (served by nginx with TLS) is the single screen an operator uses. Its seven pages map onto everything above:

Page	Shows
Overview	Zone health, model status, the live threat score — the calm baseline.
AI Engine	Model details and the attack-injection control used in the demo.
PLC Control	Live coils/registers, and start/stop/E-stop/reset buttons.
Security	Anomaly alerts with source IP and the top contributing features.
Stages	Vulnerability inventory, CVE correlation, baseline drift, integrity, pipeline verdict.
Vendor	Create and audit time-boxed vendor sessions.
Incidents	Open incidents, the steps that ran automatically, and any pending approvals.

End of Part 3. You can now narrate the entire brain: traffic → 20 features → four ML models across two planes (network + robot motion) → alert → graded incident response, plus vulnerability management, the six-gate CI/CD pipeline, monitoring, vendor access, and the dashboard. **Part 4** turns all of this into performance: how to run it, the exact live demo, the questions you will be asked, and the honest trade-offs that win senior-level credibility.